



Database Management Systems

Join expressions

Anchor:

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL in Database Design

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

- Join operations take two relations and return as a result another relation.
- A join operation is a Cartesian product that requires tuples in the two relations to match (under some conditions).
- It also specifies the attributes that are present in the result of the join
- The join operations are typically used as subquery expressions in the **FROM** clause

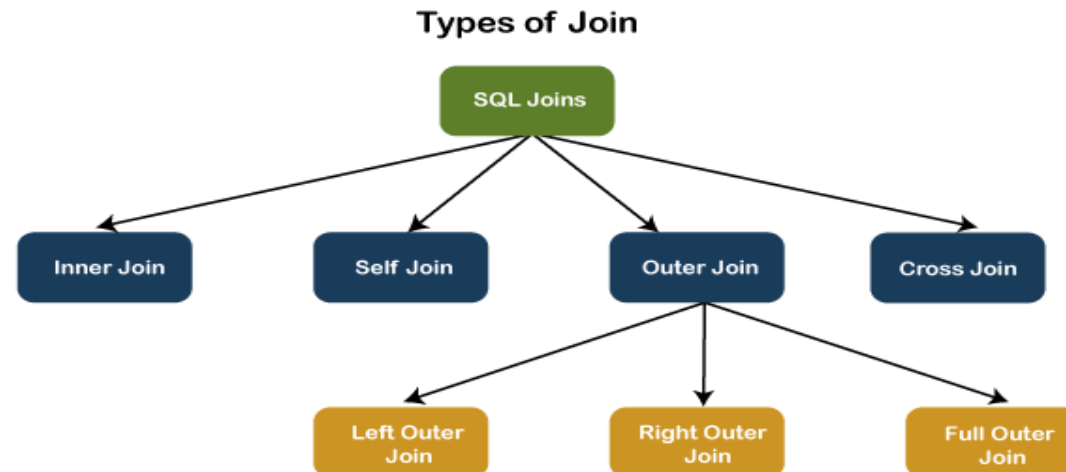
Three types of joins:

Natural join

Inner join

Outer join

Cross join



- **A natural join is a type of join operation that creates an implicit join by combining tables based on columns with the same name and data type.**
- In a NATURAL JOIN on two relations R and S, no join condition is specified.
- An implicit EQUIJOIN condition for each pair of attributes with the same name from R and S is created.
- Each such pair of attributes is included only once in the resulting relation.
- If the names of the join attributes are not the same in the base relations, it is possible to rename the attributes so that they match, and then to apply NATURAL JOIN.

Points to remember:

- There is no need to specify the column names for natural join.
- The resultant table always contains unique columns.
- It is possible to perform a natural join on more than two tables.
- Do not use the ON clause.

The following is a basic syntax to illustrate the natural join:

```
SELECT [column_names | *]  
FROM table_name1  
NATURAL JOIN table_name2;
```

Database Management Systems

Natural Join



Employee Table:

Employee_ID	employee_name	employee_salary
1	Arun Tiwari	50000
2	Sachin Rathi	64000
3	Harshal Pathak	48000
4	Arjun Kuwar	46000
5	Sarthak Gada	62000
6	Saurabh Sheik	53000
7	Shubham Singh	29000
8	Shivam Dixit	54000
9	Vicky Gujral	39000
10	Vijay Bose	28000
NULL	NULL	NULL

Department	Department_name	Employee_ID
1	Production	1
2	Sales	3
3	Marketing	4
4	Accounts	5
5	Development	7
6	HR	9
7	Sales	10
NULL	NULL	NULL

Loan:

loan_id	branch	amount
1	B1	15000
2	B2	10000
3	B3	20000
4	B4	100000
5	B5	150000
6	B6	50000
7	B7	35000
8	B8	85000
NULL	NULL	NULL

CustID	CustName	loan_id
1	Sonakshi Dixit	1
2	Shital Garg	4
3	Swara Joshi	5
4	Isha Deshmukh	2
5	Swati Bose	7
6	Asha Kapoor	10
7	Nandini Shah	9
NULL	NULL	NULL

Example: List the employees along with their department names.

Query: SELECT employee_name, employee_salary, Department_name
FROM employee NATURAL JOIN Department;

Output:

employee_name	employee_salary	Department_name
Arun Tiwari	50000	Production
Harshal Pathak	48000	Sales
Arjun Kuwar	46000	Marketing
Sarthak Gada	62000	Accounts
Shubham Singh	29000	Development
Vicky Gujral	39000	HR
Vijay Bose	28000	Sales

Example: List the customers along with their loan amount and address.

Query: SELECT CustName, CustID, amount, mobile ,address
FROM Borrower NATURAL JOIN loan NATURAL JOIN cust_info ;

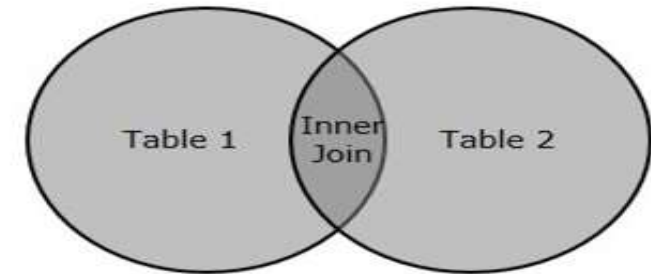
Output:

CustName	CustID	amount	mobile	address
Sonakshi Dixit	1	15000	598675498654	777 Brockton Avenue, Abington MA 251
Shital Garg	2	100000	698853747888	337 Russell St, Hadley MA 103
Swara Joshi	3	150000	234456977555	20 Soojian Dr, Leicester MA 154
Isha Deshmukh	4	10000	987656789666	780 Lynnway, Lynn MA 19
Swati Bose	5	35000	756489372222	700 Oak Street, Brockton MA 23

Customer info table:

CustID	mobile	address
1	598675498654	777 Brockton Avenue, Abington MA 251
2	698853747888	337 Russell St, Hadley MA 103
3	234456977555	20 Soojian Dr, Leicester MA 154
4	987656789666	780 Lynnway, Lynn MA 19
5	756489372222	700 Oak Street, Brockton MA 23

- SQL Inner Join is a type of join that is used to combine records from two related tables, based and select the rows depending on the condition (join predicate)
- Inner Join query compares each row of the first table with each row of the second table to find all pairs of rows that satisfy the join-predicate.
- When the join-predicate is satisfied, column values for each matched pair of rows of both tables are combined into a result row.
- The Inner Join is a default join; i.e., even if the “Join” keyword is used instead of “Inner Join”, tables are joined using matching records of common columns, by default.
- The most important and frequently used join in SQL is the Inner Join. It is also referred as Equijoin.



Example : Write a query to perform inner join considering employee table and department table.

Query:

```
SELECT e.Employee_ID, e.Employee_Name, e.Employee_Salary, d.DepartmentID,  
d.Department_Name FROM employee e INNER JOIN department d ON e.Employee_ID =  
d.Employee_ID;
```

Output:

Employee_ID	Employee_Name	Employee_Salary	DepartmentID	Department_Name
1	Arun Tiwari	50000	1	Production
3	Harshal Pathak	48000	2	Sales
4	Arjun Kuwar	46000	3	Marketing
5	Sarthak Gada	62000	4	Accounts
7	Shubham Singh	29000	5	Development
9	Vicky Gujral	39000	6	HR
10	Vijay Bose	28000	7	Sales

Example Write a query to perform inner join considering loan table and borrower table.

Query:

```
SELECT l.Loan_ID, l.Branch, l.Amount, b.CustID, b.CustName FROM Loan l JOIN  
Borrower b ON l.Loan_ID = b.Loan_ID;
```

Output:

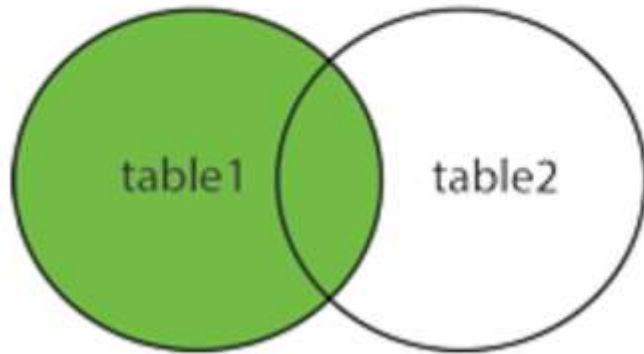
Loan_ID	Branch	Amount	CustID	CustName
1	B1	15000	1	Sonakshi Dixit
4	B4	100000	2	Shital Garg
5	B5	150000	3	Swara Joshi
2	B2	10000	4	Isha Deshmukh
7	B7	35000	5	Swati Bose

- An extension of the join operation that avoids loss of information in few cases.
- Computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join.
- In the SQL outer JOIN, all the content from both the tables is integrated together.
- Even though the records from both the tables are matched or not, the matching and non-matching records from both the tables will be considered an output of the outer join in SQL.

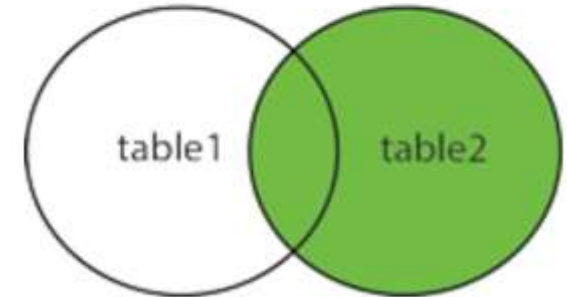
There are three different types of outer join in SQL:

Left Outer Join
Right Outer Join
Full Outer Join

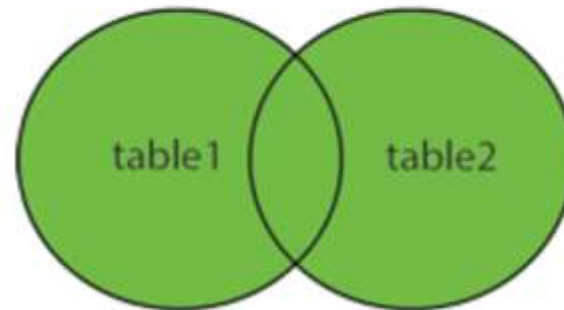
LEFT JOIN



RIGHT JOIN



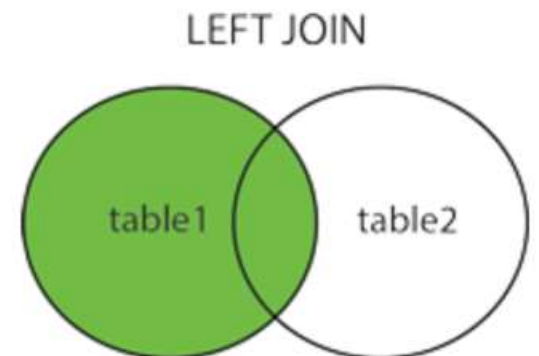
FULL OUTER JOIN



- In LEFT OUTER JOIN every tuple in the left table must appear in the result. If it does not have a matching tuple, it is padded with NULL values for the attributes of the right table.
- If we use the left outer join to combine two different tables, then we will get all the records from the left table.
- But we will get only those records from the right table, which have the corresponding key in the left table ie the matching tuple

Syntax:

```
SELECT TableName1.columnName1, TableName2.columnName2  
FROM TableName1 LEFT OUTER JOIN TableName2 ON  
TableName1.ColumnName = TableName2.ColumnName;
```



Example : Write a query to perform left outer join considering employee table as the left table and department table as the right table.

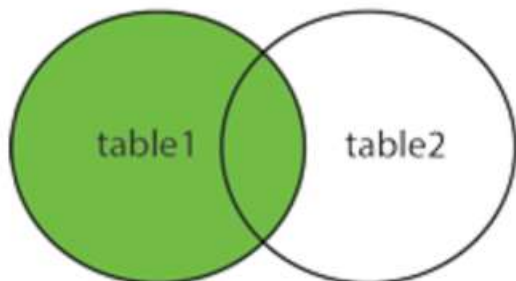
Query:

```
SELECT    e.Employee_ID,    e.Employee_Name,    e.Employee_Salary,    d.DepartmentID,  
d.Department_Name FROM employee e LEFT OUTER JOIN department d ON e.Employee_ID =  
d.Employee_ID;
```

Output:

Employee_ID	Employee_Name	Employee_Salary	DepartmentID	Department_Name
1	Arun Tiwari	50000	1	Production
2	Sachin Rathi	64000	NULL	NULL
3	Harshal Pathak	48000	2	Sales
4	Arjun Kuwar	46000	3	Marketing
5	Sarthak Gada	62000	4	Accounts
6	Saurabh Sheik	53000	NULL	NULL
7	Shubham Singh	29000	5	Development
8	Shivam Dixit	54000	NULL	NULL
9	Vicky Gujral	39000	6	HR
10	Vijay Bose	28000	7	Sales

LEFT JOIN

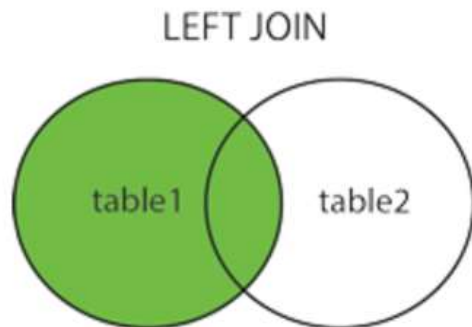


Example Write a query to perform left outer join considering loan table as the left table and borrower table as the right table.

Query:

```
SELECT L.Loan_ID, L.Branch, L.Amount, B.CustID, B.CustName FROM Loan L LEFT  
OUTER JOIN Borrower B ON L.Loan_ID = B.Loan_ID;
```

Output:

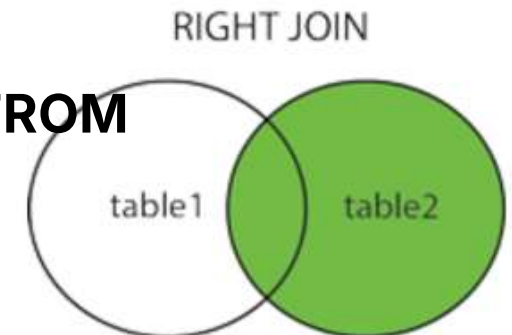


Loan_ID	Branch	Amount	CustID	CustName
1	B1	15000	1	Sonakshi Dixit
2	B2	10000	4	Isha Deshmukh
3	B3	20000	NULL	NULL
4	B4	100000	2	Shital Garg
5	B5	150000	3	Swara Joshi
6	B6	50000	NULL	NULL
7	B7	35000	5	Swati Bose
8	B8	85000	NULL	NULL

- In RIGHT OUTER JOIN every tuple in the right table must appear in the result. If it does not have a matching tuple, it is padded with NULL values for the attributes of the left table. Right outer join is the reverse of the left outer join.
- If we use the right outer join to combine two different tables, then we will get all the records from the right table.
- But we will get only those records from the left table, which have the corresponding key in the right table.

Syntax :

```
SELECT TableName1.columnName1, TableName2.columnName2 FROM  
TableName1 RIGHT OUTER JOIN TableName2 ON  
TableName1.ColumnName = TableName2.ColumnName;
```

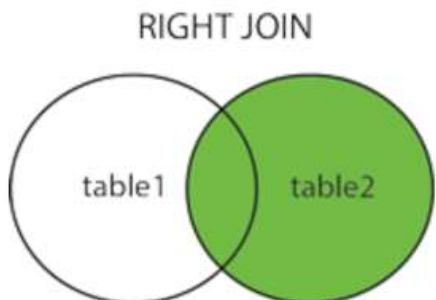


Example:

Write a query to perform right outer join considering employee table as the left table and department table as the right table.

Query:

```
SELECT    e.Employee_ID,    e.Employee_Name,    e.Employee_Salary,    d.DepartmentID,  
d.Department_Name FROM employee e RIGHT OUTER JOIN department d ON e.Employee_ID =  
d.Employee_ID;
```



Output:

Employee_ID	Employee_Name	Employee_Salary	DepartmentID	Department_Name
1	Arun Tiwari	50000	1	Production
3	Harshal Pathak	48000	2	Sales
4	Arjun Kuwar	46000	3	Marketing
5	Sarthak Gada	62000	4	Accounts
7	Shubham Singh	29000	5	Development
9	Vicky Gujral	39000	6	HR
10	Vijay Bose	28000	7	Sales

Example:

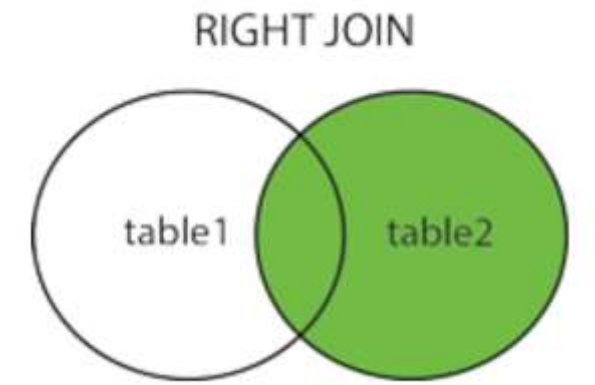
Write a query to perform right outer join considering the loan table as the left table and the borrower table as the right table.

Query:

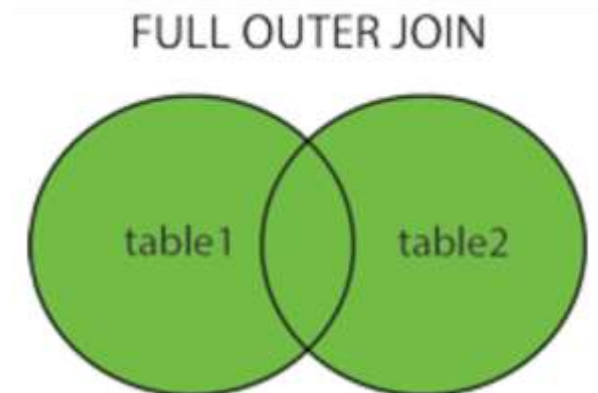
```
SELECT l.Loan_ID, l.Branch, l.Amount, b.CustID, b.CustName
FROM Loan l RIGHT OUTER JOIN Borrower b ON l.Loan_ID = b.Loan_ID;
```

Output:

Loan_ID	Branch	Amount	CustID	CustName
1	B1	15000	1	Sonakshi Dixit
4	B4	100000	2	Shital Garg
5	B5	150000	3	Swara Joshi
2	B2	10000	4	Isha Deshmukh
7	B7	35000	5	Swati Bose
NULL	NULL	NULL	6	Asha Kapoor
NULL	NULL	NULL	7	Nandini Shah



- If we use a full outer join to combine two different tables, then we will get all the records from both tables.
- We will get all the records from the left table as well as the right table.
- **MySQL doesn't support FULL OUTER JOIN directly.**
- So to implement full outer join in MySQL, we will execute two queries in a single query.
- The first query will be of LEFT OUTER JOIN, and the second query will be of RIGHT OUTER JOIN.
- We will combine the first and the second query with the UNION operator to see the results of FULL OUTER JOIN.



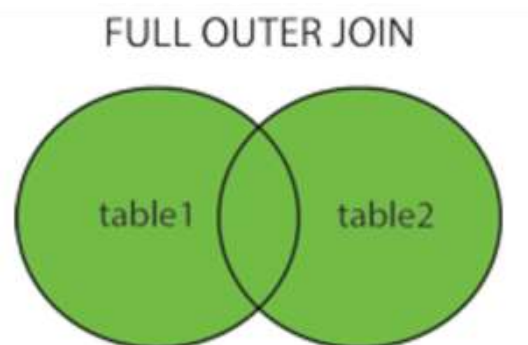
Example 1:

Write a query to perform full outer join considering the employee table as the left table and department table as the right table.

Query: `SELECT e.Employee_ID, e.Employee_Name, e.Employee_Salary, d.DepartmentID, d.Department_Name FROM employee e LEFT OUTER JOIN department d ON e.Employee_ID = d.Employee_ID UNION SELECT e.Employee_ID, e.Employee_Name, e.Employee_Salary, d.DepartmentID, d.Department_Name FROM employee e RIGHT OUTER JOIN department d ON e.Employee_ID = d.Employee_ID;`

Output:

Employee_ID	Employee_Name	Employee_Salary	DepartmentID	Department_Name
1	Arun Tiwari	50000	1	Production
3	Harshal Pathak	48000	2	Sales
4	Arjun Kuwar	46000	3	Marketing
5	Sarthak Gada	62000	4	Accounts
7	Shubham Singh	29000	5	Development
9	Vicky Gujral	39000	6	HR
10	Vijay Bose	28000	7	Sales
2	Sachin Rathni	64000	NULL	NULL
6	Saurabh Sheik	53000	NULL	NULL
8	Shivam Dixit	54000	NULL	NULL



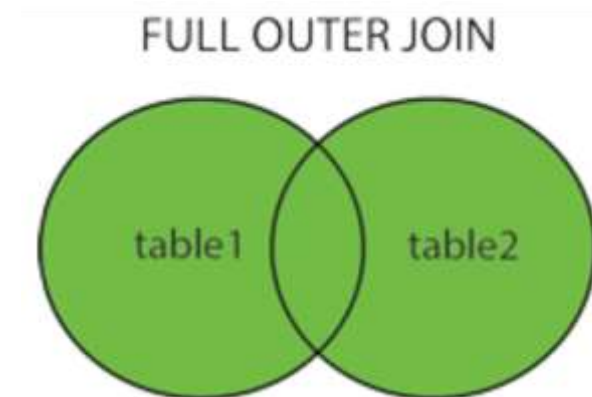
Example 1:

Write a query to perform full outer join considering loan table as the left table and borrower table as the right table.

Query: SELECT l.Loan_ID, l.Branch, l.Amount, b.CustID, b.CustName FROM Loan l LEFT OUTER JOIN Borrower b ON l.Loan_ID = b.Loan_ID UNION SELECT l.Loan_ID, l.Branch, l.Amount, b.CustID, b.CustName FROM Loan l RIGHT OUTER JOIN Borrower b ON l.Loan_ID = b.Loan_ID;

Output:

Loan_ID	Branch	Amount	CustID	CustName
1	B1	15000	1	Sonakshi Dixit
2	B2	10000	4	Isha Deshmukh
3	B3	20000	NULL	NULL
4	B4	100000	2	Shital Garg
5	B5	150000	3	Swara Joshi
6	B6	50000	NULL	NULL
7	B7	35000	5	Swati Bose
8	B8	85000	NULL	NULL
NULL	NULL	NULL	6	Asha Kapoor
NULL	NULL	NULL	7	Nandini Shah



- Cross Join returns the Cartesian Product
- It does not use a join condition (ON ...), so it doesn't care about matching columns.
- **For example**, consider the tables loan (having 8 rows) and borrower (having 7 rows)
- Rows Returned: (Rows in Table A) * (Rows in Table B) = $8 \times 7 = 56$
- **Query: select * from loan cross join borrower;**

```
mysql> select *  
      -> from loan  
      -> cross join borrower;  
+-----+-----+-----+-----+-----+-----+  
| loan_id | branch | amount | CustID | CustName      | loan_id |  
+-----+-----+-----+-----+-----+-----+  
|      1 | B1     | 15000  |      7 | Nandini Shah  |      9 |  
|      1 | B1     | 15000  |      6 | Asha Kapoor   |     10 |  
+-----+-----+-----+-----+-----+-----+  
|      8 | B8     | 85000  |      4 | Isha Deshmukh |      2 |  
|      8 | B8     | 85000  |      3 | Swara Joshi   |      5 |  
|      8 | B8     | 85000  |      2 | Shital Garg   |      4 |  
|      8 | B8     | 85000  |      1 | Sonakshi Dixit|      1 |  
+-----+-----+-----+-----+-----+-----+  
56 rows in set (0.00 sec)
```

1) Which of the following statements is true regarding a NATURAL JOIN?

- A) It requires an explicit ON clause to match columns.
- B) Each pair of matching attributes is included twice in the resulting relation.
- C) It creates an implicit EQUIJOIN condition for each pair of attributes with the same name and data type.
- D) It pads non-matching records with NULL values.

2) Fill in the blank: The INNER JOIN is the default join in SQL, meaning if you just use the "Join" keyword, tables are joined using matching records of common columns by default. It is also commonly referred to as an _____.

3) When performing a LEFT OUTER JOIN, what happens to a tuple in the left table if it does not have a matching tuple in the right table?

- A) It is completely omitted from the result.
- B) It causes the query to return an empty set.
- C) It is padded with NULL values for the attributes of the right table.
- D) It drops the unmatched columns from the final output.

4) Fill in the blank: Because MySQL does not support the FULL OUTER JOIN keyword directly, it must be implemented by executing a LEFT OUTER JOIN and a RIGHT OUTER JOIN and combining them using the _____ operator.

5) A CROSS JOIN returns the Cartesian Product of two tables. If a loan table has 8 rows and a borrower table has 7 rows, how many rows will be returned by executing a CROSS JOIN between them?

- A) 8
- B) 15
- C) 56
- D) 0

6) Fill in the blank: Unlike inner and outer joins, a CROSS JOIN does not care about matching columns because it does not use a _____ condition (such as the ON clause).

Answers

1. C
2. Equijoin
3. C
4. UNION
5. C
6. join



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu